

Helm Deployment Guide for Moodle on Kubernetes (GitLab CE + Single VPS Cluster)

Helm Deployment Guide for Moodle on Kubernetes

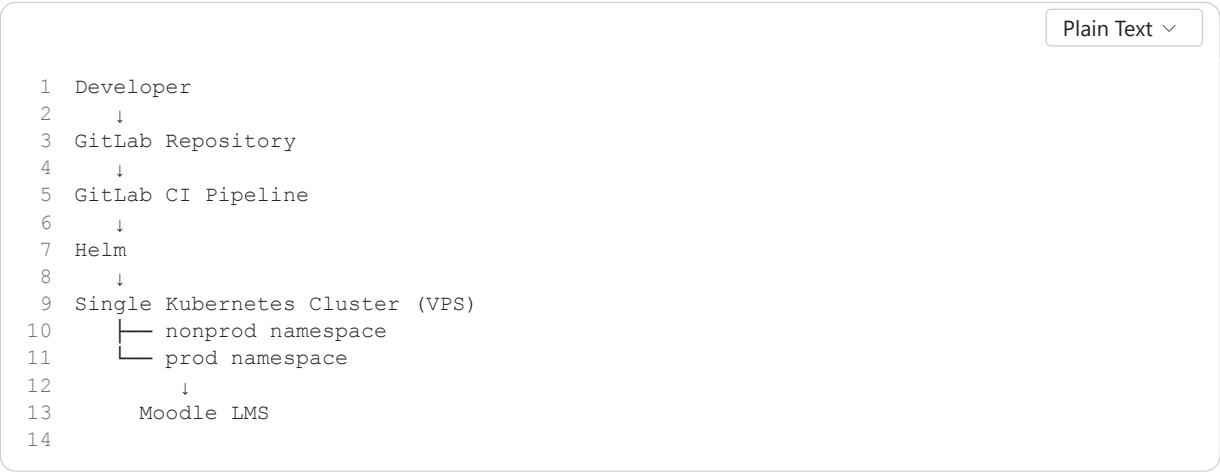
Using GitLab Community Edition and a Single VPS Kubernetes Cluster

This document is a **complete, end-to-end guide** to deploying Moodle LMS on Kubernetes using Helm, with:

- **GitLab Community Edition** (CI/CD instead of Azure DevOps)
- A **self-managed VPS** running Kubernetes
- **Non-Production and Production namespaces** in the same cluster
- Optional **open-source ITSM / Change Management** integration

The guide is written to be **practical, reproducible, and production-oriented**.

1. Target Architecture (High Level)



2. Kubernetes Setup on a VPS

2.1 Assumptions

- 1–3 VPS nodes (Ubuntu 22.04 recommended)
- Kubernetes installed using:
 - kubeadm **OR**
 - lightweight distro (k3s is acceptable for VPS)
- Storage available via:
 - hostPath (non-prod)
 - NFS / Longhorn / local-path-provisioner (prod)

2.2 Create Namespaces

Both environments live safely in **one cluster**, separated by namespaces.

Shell ▾

```
1 kubectl create namespace moodle-nonprod
2 kubectl create namespace moodle-prod
3
```

✔ Logical isolation ✔ Lower cost ✔ Same Kubernetes API behavior

2.3 Basic Namespace Hardening (Recommended)

Limit ranges (example)

YAML 

```
1  apiVersion: v1
2  kind: LimitRange
3  metadata:
4    name: default-limits
5    namespace: moodle-prod
6  spec:
7    limits:
8      - default:
9          cpu: "1"
10         memory: "2Gi"
11       defaultRequest:
12         cpu: "500m"
13         memory: "1Gi"
14       type: Container
15
```

Apply similarly (lighter) for non-prod.

3. Helm Overview (Moodle Context)

Helm acts as the **application package manager** for Kubernetes.

For Moodle, Helm manages:

- Deployments
- Services
- PVCs
- Secrets
- Database sub-chart
- Upgrades & rollbacks

- ✔ One command replaces dozens of YAML files

4. Helm Chart Selection

We use the **Bitnami Moodle Helm chart** because it is:

- Production tested
- Actively maintained
- Fully configurable via values files
- Bundles Moodle + database

5. Helm Values Structure (Non-Prod vs Prod)

5.1 Non-Production Values

values-nonprod.yaml

```
1 replicaCount: 1
2
3 image:
4   tag: 4.4
5
6 moodle:
7   username: admin
8   password: admin123
9
10 persistence:
11   enabled: true
12   size: 10Gi
13
14 mariadb:
15   enabled: true
16   auth:
17     rootPassword: root123
18     database: moodle
19
20 ingress:
21   enabled: true
22   hostname: moodle-nonprod.example.com
23
```

5.2 Production Values

values-prod.yaml

```
1 replicaCount: 3
2
3 image:
4   tag: 4.4
5
6 moodle:
7   username: admin
8   password: STRONG_PASSWORD
9   email: admin@example.com
10
11 persistence:
12   enabled: true
13   size: 50Gi
14
15 mariadb:
16   enabled: true
17   auth:
18     rootPassword: STRONG_DB_PASSWORD
19     database: moodle
20
21 ingress:
22   enabled: true
23   hostname: moodle.example.com
24
```

✅ Same chart ✅ Different risk profile

6. Deploying Moodle Manually (Baseline)

Shell ▾

```
1 helm repo add bitnami https://charts.bitnami.com/bitnami
2 helm repo update
3
4 helm upgrade --install moodle-nonprod bitnami/moodle \
5   --namespace moodle-nonprod \
6   --values values-nonprod.yaml
7
8 helm upgrade --install moodle-prod bitnami/moodle \
9   --namespace moodle-prod \
10  --values values-prod.yaml
11
```

7. GitLab Community Edition CI/CD Integration

7.1 GitLab Runner

- Install **GitLab Runner** on the VPS or a management node
- Use the **shell** or **docker** executor
- Configure kubeconfig access

7.2 Repository Layout

Plain Text ▾

```
1 repo/
2 |─ helm/
3 |   |─ values-nonprod.yaml
4 |   |─ values-prod.yaml
5 |─ .gitlab-ci.yml
6
```

7.3 GitLab CI Pipeline Example

YAML ▾

```
1 stages:
2   - validate
3   - deploy-nonprod
4   - deploy-prod
5
6 variables:
7   HELM_RELEASE: moodle
8
9 validate:
10  stage: validate
11  script:
12    - helm lint bitnami/moodle
13
14 nonprod-deploy:
15  stage: deploy-nonprod
```

```
16 script:
17   - helm upgrade --install ${HELM_RELEASE}-nonprod bitnami/moodle \
18     --namespace moodle-nonprod \
19     --values helm/values-nonprod.yaml
20
21 prod-deploy:
22   stage: deploy-prod
23   script:
24     - helm upgrade --install ${HELM_RELEASE}-prod bitnami/moodle \
25       --namespace moodle-prod \
26       --values helm/values-prod.yaml
27   when: manual
28
```

✔ Manual approval for production

8. Helm Rollbacks in GitLab

Shell ▾

```
1 helm history moodle-prod -n moodle-prod
2 helm rollback moodle-prod 1 -n moodle-prod
3
```

✔ Fast recovery ✔ Auditable via GitLab job logs

9. Helm vs Kustomize (Practical Comparison)

	≡ Topic	≡ Helm	≡ Kustomize
1	Purpose	Package & lifecycle	Config overlays
2	Stateful Apps	Excellent	Weak
3	Rollback	Built-in	Manual
4	CI/CD Fit	Strong	Moderate
5	Moodle Suitability	✔ Best choice	✗ Not ideal

Recommendation:

- Use **Helm** for Moodle
- Optionally layer **Kustomize** for cluster-specific tweaks

10. Optional: Open-Source ITSM / Change Management Integration

10.1 Typical Tools

- iTop
- GLPI
- Znuny (OTRS fork)

10.2 Change Flow

Plain Text ▾

```
1  GitLab Pipeline Start
2      ↓
3  Create Change via REST API
4      ↓
5  Wait for Approval (manual job)
6      ↓
7  Helm Deploy
8      ↓
9  Update Change to Implemented
10
```

10.3 Example Conceptual Step

Shell ▾

```
1  curl -X POST https://itsm/api/changes \
2      -H "Authorization: Bearer TOKEN" \
3      -d '{"title":"Moodle Production Release","version":"4.4"}'
4
```

11. Security & Best Practices

- Separate namespaces for env isolation
- Resource limits per namespace
- Secrets managed via Kubernetes secrets or sealed-secrets
- Manual promotion to production
- Helm rollback always enabled

12. Final Summary

Using GitLab CE + Helm on a single VPS Kubernetes cluster allows you to safely run non-prod and prod Moodle environments with strong separation, low cost, and full deployment control—without relying on managed cloud platforms.

Possible Next Expansions

- GitOps with ArgoCD
- External database for prod
- Backup & restore strategy
- TLS + cert-manager

Tell me what you want to extend next.